

Reshape, View, Permute & Transpose, Worked Out by Hand

Re-reading the same bytes through new strides

A complete numerical walk-through of the four shape-changing operations. We take one $2 \times 3 \times 4$ block of integers $0, 1, \dots, 23$ and watch what each operation does to its (shape, stride) recipe — and, crucially, what it does *not* do to the 24 numbers in memory. `reshape/view` re-letter the same flat run; `permute/transpose` merely *reorder* the strides. Every stride, offset and contiguity verdict below was produced by the reference program in Appendix A, so the arithmetic is exact.

What this document does. It fixes the running tensor to `arange(24).reshape(2,3,4)` — so the value stored at every slot *is* its own memory offset — and then derives, by hand, the new strides for `reshape(6,4)`, `(4,6)`, `(24,)`, for `transpose(1,2)`, and for `permute(2,0,1)`. For each we re-evaluate the index→offset equation, verify two entries against the stored values, and run the contiguity test.

How to read this. Each operation gets its own worked section with exact numbers, plus a coloured box isolating the one idea that trips people up. The payoff is Section 6: *why* `.view()` crashes on a transposed tensor while `.reshape()` silently succeeds — the single most common shape bug in the course.

Concepts covered, in order: the flat-line + recipe model · `reshape/view` as pure pointer re-lettering · the $s_a = \prod_{b>a} d_b$ stride rule re-applied · `transpose(1,2)` as a stride swap · `permute(2,0,1)` as a stride permutation · the invariant offset $= \sum_a s_a \text{idx}_a$ surviving every op · the contiguity test $s_a \stackrel{?}{=} \prod_{b>a} d_b$ · why `view` demands contiguity and `reshape` does not.

1 The setup: one flat line, many recipes

A tensor is one contiguous run of numbers in memory plus a *recipe* — the pair (shape, strides) — that says how to read that run as an n -dimensional box. Our run is the 24 integers $0, 1, \dots, 23$, and the starting recipe is shape $(2, 3, 4)$ with strides $(12, 4, 1)$:

$$T[0] = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{bmatrix}, \quad T[1] = \begin{bmatrix} 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 \end{bmatrix}.$$

The contiguous stride of an axis is the product of all axis sizes *after* it:

$$s_a = \prod_{b>a} d_b \quad s_2 = 1, \quad s_1 = d_2 = 4, \quad s_0 = d_1 d_2 = 12.$$

Because we filled the tensor with `arange(24)`, the value at logical index (i, j, k) equals its own flat offset:

$$\text{offset}(i, j, k) = \sum_a s_a \text{idx}_a = 12i + 4j + k$$

so $T[i, j, k] = \text{offset}(i, j, k)$. Every claim below is therefore self-checking: if our derived stride is right, plugging an index into $\sum_a s_a \text{idx}_a$ must land on the number actually stored there.

Intuition

There are exactly two things any of the four operations can do: *re-letter* the flat run (assign a new (shape, stride) that still walks the bytes in their stored order) or *re-order* the strides (keep the bytes put, but change which logical step skips how far). `reshape/view` do the first; `permute/transpose` do the second. Nobody moves the 24 numbers unless contiguity is later forced.

2 Step 1 — reshape/view: re-lettering the same run

`reshape` and `view` ask: *can I read this exact flat line, in its exact stored order, as a box of the new shape?* If the element count matches ($\prod$ of new shape = 24) and the layout is contiguous, the answer is yes and the cost is zero — only the recipe changes. The new strides come from the *same* rule $s_a = \prod_{b>a} d_b$ applied to the new shape.

$(2, 3, 4) \rightarrow (6, 4)$. Last axis packed $\Rightarrow s_1 = 1$; one row jumps a whole line of 4 $\Rightarrow s_0 = 4$. New strides $(4, 1)$. The flat line is untouched; row r , column c now reads offset $4r + c$.

$(2, 3, 4) \rightarrow (4, 6)$. $s_1 = 1$, $s_0 = d_1 = 6$. New strides $(6, 1)$.

$(2, 3, 4) \rightarrow (24,)$. A single packed axis: $s_0 = 1$. New strides $(1,)$ — this is the raw flat line itself.

Operation	New shape	New strides	Contiguous?
<code>reshape(6, 4)</code>	$(6, 4)$	$(4, 1)$	True
<code>reshape(4, 6)</code>	$(4, 6)$	$(6, 1)$	True
<code>reshape(24)</code>	$(24,)$	$(1,)$	True

Same number, new address. Take the stored value 17. In the original (2, 3, 4) it sits at (1, 1, 1): $12 \cdot 1 + 4 \cdot 1 + 1 = 17$. After `reshape(6,4)` the strides are (4, 1), so the slot holding 17 is (r, c) with $4r + c = 17$, i.e. $r = 4, c = 1$ — and indeed $T_{6 \times 4}[4, 1] = 17$. The byte never moved; only the two-coordinate label changed.

Mini-example — this operation in isolation

`reshape` is a base-conversion of the offset, nothing more. Offset 17 in mixed radix (2, 3, 4) peels off as $17 = 12 \cdot 1 + 4 \cdot 1 + 1 \rightarrow (1, 1, 1)$; in radix (6, 4) it peels off as $17 = 4 \cdot 4 + 1 \rightarrow (4, 1)$; in radix (24,) it is just (17,). Three labels, one byte. That is the whole of `reshape`.

3 Step 2 — `transpose(1,2)`: swap two strides, move nothing

A transpose refuses to copy data. To swap axes 1 and 2 it simply swaps the corresponding entries of the *shape* and of the *stride* vector:

$$\text{shape } (2, 3, 4), \text{ strides } (12, 4, 1) \xrightarrow{\text{swap axes 1,2}} \text{shape } (2, 4, 3), \text{ strides } (12, 1, 4).$$

The bytes are exactly where they were; we have only relabelled which logical step advances by how much. Logical index (i, j, k) of the transposed tensor now maps to

$$\text{offset}(i, j, k) = 12i + 1 \cdot j + 4 \cdot k = 12i + j + 4k.$$

Verify two entries against storage.

$$\begin{aligned} \text{offset}(1, 3, 2) &= 12 \cdot 1 + 1 \cdot 3 + 4 \cdot 2 = 23 & T_{\text{flat}}[23] &= 23 \checkmark \\ \text{offset}(0, 2, 1) &= 12 \cdot 0 + 1 \cdot 2 + 4 \cdot 1 = 6 & T_{\text{flat}}[6] &= 6 \checkmark \end{aligned}$$

Both hit the value stored at that flat slot, confirming the swapped strides are correct without anything being copied.

Intuition

The transposed tensor's logical element (i, j, k) is the original element (i, k, j) — you read the same number, you just reach it by a different walk. The stride swap is precisely what makes "look up (i, k, j) " and "look up (i, j, k) with swapped strides" the same arithmetic.

4 Step 3 — `permute(2,0,1)`: permute all the strides

`permute` generalises transpose to *all* axes at once. `permute(2,0,1)` says "the new axis 0 is the old axis 2, new axis 1 is old axis 0, new axis 2 is old axis 1." Apply that permutation to both the shape and the strides:

$$\begin{aligned} \text{shape: } (d_2, d_0, d_1) &= (4, 2, 3), \\ \text{strides: } (s_2, s_0, s_1) &= (1, 12, 4). \end{aligned}$$

So shape (4, 2, 3), strides (1, 12, 4) — again zero data movement. The new addressing equation is

$$\text{offset}(i, j, k) = 1 \cdot i + 12 \cdot j + 4 \cdot k = i + 12j + 4k.$$

Verify two entries.

$$\begin{aligned}\text{offset}(1, 0, 2) &= 1 \cdot 1 + 12 \cdot 0 + 4 \cdot 2 = 9 & T_{\text{flat}}[9] &= 9 \checkmark \\ \text{offset}(3, 1, 2) &= 1 \cdot 3 + 12 \cdot 1 + 4 \cdot 2 = 23 & T_{\text{flat}}[23] &= 23 \checkmark\end{aligned}$$

Watch out

`permute` takes the *destination order of the old axes*, not a pairwise swap. `permute(2,0,1)` is *not* "swap 2 and 0"; it cyclically rotates the axis labels. A frequent bug is permuting an image back with the same tuple you permuted it with: $(C,H,W) \rightarrow \text{permute}(1,2,0) \rightarrow (H,W,C)$, but the inverse is `permute(2,0,1)`, *not* `permute(1,2,0)` again.

5 Step 4 — The contiguity test, axis by axis

A tensor is **contiguous** exactly when its current strides still satisfy the packed-layout rule for its current shape:

$$\text{contiguous} \iff s_a = \prod_{b>a} d_b \text{ for every axis } a$$

The reshapes pass this trivially (we built their strides from the rule). The reordering operations fail it. Run the test on `transpose(1,2)`: shape $(2, 4, 3)$, strides $(12, 1, 4)$.

axis a	stride s_a	$\prod_{b>a} d_b$	match?
0	12	$4 \cdot 3 = 12$	yes
1	1	3	no
2	4	1	no

Axis 1's stride is 1 but the rule demands 3; the fast axis (stride 1) is no longer the *last* axis, so reading the tensor in logical order jumps around the flat line. A single failing row makes the whole tensor non-contiguous. The same failure happens for `permute(2,0,1)` (strides $(1, 12, 4)$ vs the required $(6, 3, 1)$).

Intuition

Contiguity is not about whether the data is "messy" — it is the single yes/no question "does walking the box in logical order also walk memory in increasing order, with no gaps and no jumps?" `reshape/view` preserve that walk; `transpose/permute` scramble it because they move the stride-1 axis away from the last position.

6 Step 5 — The one invariant: $\text{index} \cdot \text{stride} = \text{offset}$

Across all four operations, one equation never changes form:

$$\text{offset} = \sum_a s_a \text{idx}_a$$

Only the (shape, stride) plugged into it changes. The flat data — and therefore the multiset of reachable offsets $\{0, 1, \dots, 23\}$ — is identical in every case. Here is the same handful of stored values, reached through four different recipes:

Recipe (shape; strides)	Index	$\sum s_a \text{idx}_a$	stored value
(2, 3, 4); (12, 4, 1)	(1, 1, 1)	$12+4+1 = 17$	17
(6, 4); (4, 1)	(4, 1)	$16+1 = 17$	17
<code>transpose(1,2)</code> (2, 4, 3); (12, 1, 4)	(1, 1, 1)	$12+1+4 = 17$	17
<code>permute(2,0,1)</code> (4, 2, 3); (1, 12, 4)	(2, 1, 1)	$2+12+4 = 18$	18

The first three rows reach the *same* byte (17) by three different index labels — they are three views of one storage. This is why a `view` shares memory: writing through any of these labels mutates the single underlying 17.

7 Step 6 — Why `.view()` crashes after a transpose

`.view()` is the zero-copy path: it *insists* the new shape be readable from the existing flat line in stored order, which requires (compatible) contiguous layout. On a fresh contiguous tensor that always holds, so `.view()` succeeds. After `transpose(1,2)` the strides are (12, 1, 4) — non-contiguous — so the 24 numbers, walked in the new logical order, do *not* form one packed run. There is no honest stride vector for the requested flat shape, so:

```
x_t.view(-1) ==> RuntimeError: view size is not compatible with
input tensor's size and stride.
```

`.reshape()` makes no such promise. It tries the free `view` path first; if the layout forbids it, it *silently copies* the data into a fresh contiguous buffer and views that. So `x_t.reshape(-1)` always works — it just pays a copy when it must. The explicit equivalent is `x_t.contiguous().view(-1)`: `.contiguous()` performs the copy (writing new packed strides), after which `.view()` is legal again.

Watch out

The error "view size is not compatible with input tensor's size and stride" almost always means *you called `.view()` on the output of a `permute/transpose`*. Two fixes: (1) replace `.view(...)` with `.reshape(...)`; (2) insert `.contiguous()` before `.view(...)`. Both copy; `.reshape` just hides it.

When this is the right choice

Reach for `.view()` when you have *verified* the tensor is contiguous and you want a guarantee that no copy happens (e.g. a hot loop that must share storage with its parent). Otherwise default to `.reshape()`: it is always safe, and on already-contiguous input it is exactly as cheap as `.view()` — the copy only fires when the layout genuinely requires it.

8 The four operations, at a glance

Operation	Shape (2, 3, 4) →	Strides (12, 4, 1) →	Contig. after?	Copies data?
<code>reshape(6,4)</code>	(6, 4)	(4, 1)	True	no
<code>reshape(4,6)</code>	(4, 6)	(6, 1)	True	no
<code>reshape(24)</code>	(24,)	(1,)	True	no
<code>transpose(1,2)</code>	(2, 4, 3)	(12, 1, 4)	False	no
<code>permute(2,0,1)</code>	(4, 2, 3)	(1, 12, 4)	False	no
<code>.view(...)</code>	any compat.	rule strides	needs True	no (or errors)
<code>.contiguous()</code>	unchanged	re-packed	True	yes (if needed)

Read every row as the same fact: shape and strides are bookkeeping; only `.contiguous()` (and the hidden copy inside `.reshape`) ever touches the 24 numbers.

9 Equation cheat-sheet

Contiguous stride	$s_a = \prod_{b>a} d_b$ (last axis: $s_{n-1} = 1$)
Index → offset	$\text{offset} = \sum_a s_a \text{idx}_a$
<code>reshape/view</code>	new shape with $\prod d_a$ unchanged; strides = $\prod_{b>a} d_b$
<code>transpose(p,q)</code>	swap $d_p \leftrightarrow d_q$ and $s_p \leftrightarrow s_q$
<code>permute(π)</code>	new shape (d_{π_a}), new strides (s_{π_a})
Contiguity test	$s_a \stackrel{?}{=} \prod_{b>a} d_b$ for all a
view legality	requires contiguous (compatible) layout
<code>reshape</code> fallback	view if possible, else <code>contiguous().view(...)</code>

Appendix A Reference implementation

Every shape, stride, offset and contiguity verdict above is reproduced by the following program. NumPy gives the identical layout; the `.view()` crash is a PyTorch behaviour shown at the end.

```
import torch

T = torch.arange(24).reshape(2, 3, 4)
print(T.shape, T.stride(), T.is_contiguous())    # (2,3,4) (12,4,1) True

# --- reshape / view: re-letter the same flat run (free) ---
print(T.reshape(6, 4).stride())    # (4, 1)
print(T.reshape(4, 6).stride())    # (6, 1)
print(T.reshape(24).stride())      # (1,)

# --- transpose(1,2): swap two strides, move nothing ---
Tt = T.transpose(1, 2)
print(Tt.shape, Tt.stride(), Tt.is_contiguous()) # (2,4,3) (12,1,4) False

# --- permute(2,0,1): permute all strides ---
Tp = T.permute(2, 0, 1)
print(Tp.shape, Tp.stride(), Tp.is_contiguous()) # (4,2,3) (1,12,4) False

# --- the invariant: offset = sum(stride_i * idx_i) ---
flat = T.reshape(-1)
def offset(strides, idx):
    return sum(s * i for s, i in zip(strides, idx))
assert offset((12, 1, 4), (1, 3, 2)) == 23 and flat[23] == 23    # transpose
assert offset((12, 1, 4), (0, 2, 1)) == 6  and flat[6]  == 6
assert offset((1, 12, 4), (1, 0, 2)) == 9  and flat[9]  == 9    # permute
assert offset((1, 12, 4), (3, 1, 2)) == 23 and flat[23] == 23

# --- contiguity test on the transpose, axis by axis ---
shape, strides = (2, 4, 3), (12, 1, 4)
import math
for a in range(3):
    prod_later = math.prod(shape[a + 1:])
    print(a, strides[a], prod_later, strides[a] == prod_later)
# 0 12 12 True / 1 1 3 False / 2 4 1 False -> non-contiguous

# --- why view() fails after transpose, reshape() does not ---
try:
    Tt.view(-1)
except RuntimeError as e:
    print("Error:", e)          # view size is not compatible ... size and stride
print(Tt.reshape(-1).shape)    # (24,) -- copies silently
print(Tt.contiguous().view(-1).shape) # (24,) -- explicit copy then view
```

— end of document —